GRUPO 1 · REQUERIMIENTOS FUNCIONALES

Cuentas Bancarias

Módulo: Cuentas, saldos, estados y movimientos

Curso de Automatización de Pruebas de Software

Universidad Nacional de Asunción — CIT

Profesor: Steven Gracia

Campo	Valor
Documento	Requerimientos funcionales — Grupo 1
Versión	1.0
Fecha	10 de octubre de 2026
Sistemas alcanzados	aiquaa-sandbox-api (REST, `/api/v2`) y aiquaa-sandbox-web (front, `/v2`)
Fuente de la lógica	Código fuente de ambos repositorios y scripts SQL del schema qa_training_v2
Uso previsto	Base para escribir escenarios BDD/Gherkin y casos de prueba automatizados

> Documento derivado del código fuente de aiquaa-sandbox-api y aiquaa-sandbox-web. Describe el comportamiento realmente implementado en el sandbox de práctica, no un sistema bancario productivo. El anexo normativo es material de referencia orientativo.

Contenido

1. Alcance y actores
2. Precondiciones y acceso
3. Modelo de datos del módulo
4. Requerimientos funcionales
5. Reglas transversales de la API
6. Flujo en la aplicación web
7. Criterios de aceptación
8. Anexo normativo (BCP / SEDECO) y brechas

1. Alcance y actores

Cubre el ciclo de vida de una cuenta bancaria: apertura, consulta, modificación de sus datos, cambio de estado de negocio (`activa` / `bloqueada` / `cerrada`) y baja; la consulta del saldo; y el libro de movimientos de la cuenta, con el registro de depósitos (créditos) y retiros (débitos) que actualizan el saldo.

Fuera de alcance: Transferencias entre cuentas (Grupo 4), aunque sus movimientos aparecen en el extracto de la cuenta; intereses, comisiones y mantenimiento de cuenta; sobregiro (el saldo nunca puede quedar negativo); conversión de moneda; y la edición o anulación de un movimiento ya registrado.

Actores

Actor	Descripción
Alumno / QA automatizador	Consume la API con la API key de su grupo (o con un token JWT de grupo) y automatiza escenarios BDD sobre el módulo de su grupo.
Cliente del banco (fila de `usuarios`)	Identidad de dominio sobre la que operan los endpoints: es titular de cuentas, tarjetas, préstamos, beneficiarios, planes de ahorro y depósitos.
API REST (aiquaa-sandbox-api)	Ejecuta la lógica de negocio con SQL fijo sobre el schema `qa_training_v2`, bajo el rol de base de datos `qa_api` (SELECT + INSERT + UPDATE, sin DELETE).
Aplicación web (aiquaa-sandbox-web)	Interfaz tipo home banking que consume la API a través del proxy `/api/proxy/v2/{path}`; nunca llama al backend directamente desde el navegador.

Endpoints del módulo

Método y ruta	Requerimiento	Descripción
GET /api/v2/cuentas	RF-G1-01	Lista cuentas vigentes, filtradas por titular y/o estado.
POST /api/v2/cuentas	RF-G1-02	Abre una cuenta para un cliente (saldo inicial 0).
GET /api/v2/cuentas/{id}	RF-G1-03	Devuelve el detalle de una cuenta.
PUT /api/v2/cuentas/{id}	RF-G1-04	Reemplaza tipo y moneda de una cuenta.
DELETE /api/v2/cuentas/{id}	RF-G1-05	Da de baja una cuenta sin saldo.
PATCH /api/v2/cuentas/{id}/estado	RF-G1-06	Bloquea, desbloquea o cierra una cuenta.
GET /api/v2/cuentas/{id}/saldo	RF-G1-07	Vista reducida: saldo, estado y fecha del último movimiento.
GET /api/v2/cuentas/{id}/movimientos	RF-G1-08	Extracto de la cuenta, opcionalmente por tipo.
POST /api/v2/cuentas/{id}/movimientos	RF-G1-09	Registra un depósito o un retiro y actualiza el saldo.
POST /api/v2/g1/auth/token	RF-G1-TK	Emite el token JWT del grupo.

2. Precondiciones y acceso

- **Base URL de la API:** `https://aiquaa-sandbox-api.vercel.app`. La documentación interactiva del curso está en `/docs/v2` y la especificación OpenAPI en `GET /api/v2/docs`. La aplicación web está en `https://aiquaa-sandbox-web.vercel.app`.
- Toda request debe autenticarse por una de dos vías: el header `x-api-key` con una clave activa de `public.api_keys`, o el header `Authorization: Bearer <token>` con un JWT de grupo vigente. Si se envían ambos, manda `x-api-key`. Sin credenciales, o con credenciales inválidas, inactivas o vencidas, la respuesta es `401 UNAUTHORIZED` con un mensaje genérico que no indica cuál fue el motivo.
- Las rutas bajo `/api/v2` solo aceptan credenciales del **curso 2**: una API key válida pero del curso 1 recibe `403 FORBIDDEN` con el mensaje "Esta API key pertenece al curso 1 y esta ruta es del curso 2.". El gate es simétrico: una key del curso 2 tampoco sirve en `/api/v1`.
- Cada credencial admite un máximo de **30 requests por minuto** (ventana deslizante, contada por API key o por usuario del token, no por IP). Superado el límite la API responde `429 RATE_LIMITED` con los headers `Retry-After`, `X-RateLimit-Limit`, `X-RateLimit-Remaining` y `X-RateLimit-Reset`.
- Los datos de práctica viven en el schema aislado `qa_training_v2` (10 tablas) y se cargan con `scripts/seed-data-v2.sql`: 8 clientes (más un cliente por grupo, usado por el login de token), 12 cuentas, 20 movimientos, 10 tarjetas, 6 préstamos con sus cuotas, 10 beneficiarios, 12 transferencias, 6 planes de ahorro y 6 depósitos. La data inicial es determinística, pero **el sandbox es compartido por los cinco grupos**: los registros sembrados pueden haber sido modificados por otra suite. Un escenario robusto crea sus propios datos en lugar de depender del estado de una fila sembrada.
- **Clientes de soporte:** `GET|POST /api/v2/usuarios` y `GET|PUT|DELETE /api/v2/usuarios/{id}` no pertenecen a ningún grupo y están disponibles para todos; sirven para crear el titular de los productos de cada escenario. `GET /usuarios` acepta el filtro `email`; `POST` exige `nombre`, `email` y `documentoNumero` (email y documento son únicos: repetirlos da `409 CONFLICT`).
- **Sandbox SQL:** `POST /api/v2/sql/select` y `POST /api/v2/sql/update` reciben `{ "sql", "params" }` y permiten consultar o preparar datos directamente sobre las 10 tablas del curso. Admiten un único statement por request, con placeholders `$1..$n`; el `UPDATE` exige cláusula `WHERE`. Son la vía para verificar en base lo que hizo un endpoint, o para forzar un estado que la API no produce.
- La API no expone borrado físico: el rol `qa_api` no tiene permiso `DELETE` en Postgres, y todo endpoint `DELETE` hace baja lógica (`activo = false`, o `activa = false` en `cuentas`) y devuelve `204 No Content` sin cuerpo. Una fila dada de baja deja de verse en `GET`, `PUT` y `DELETE`, pero sigue existiendo en la base.
- En la aplicación web el acceso tiene tres capas, en orden: elegir el curso (`/curso`, opción 2), cargar la API key del curso (`/login`, que valida que la key sea del curso elegido) y elegir el cliente con el que se opera (`/auth/login`, por email de un cliente activo). Las pantallas `/v2/usuarios/*` son la excepción a la tercera capa: se pueden usar solo con la API key, porque son el punto de partida para crear un cliente.
- Para abrir una cuenta se necesita un cliente existente (`usuarioId`); se puede crear uno con `POST /api/v2/usuarios` o usar uno de los sembrados (ids 1 a 8).
- Las cuentas se identifican por su número interno (`id`), no por el `numero_cuenta` comercial de 10 dígitos.
- Datos sembrados útiles: las cuentas 1 a 7, 9, 11 y 12 nacen `activa` con saldo; la cuenta 8 nace `bloqueada` con saldo 0; la cuenta 10 nace `cerrada` con saldo 45.000 (un estado que la API no permite

alcanzar: ver brechas). Las cuentas 2, 7 y 12 son en dólares.

3. Modelo de datos del módulo

cuentas

Cuentas de los clientes. `estado` es el estado de negocio; `activa` es la marca de baja lógica. Son cosas distintas: bloquear o cerrar una cuenta no la da de baja.

Columna	Tipo / restricción
`id`	`bigserial`, clave primaria
`usuario_id`	`bigint`, obligatorio, referencia a `usuarios(id)`
`numero_cuenta`	`text`, obligatorio y único; lo genera `POST /cuentas` al azar (10 dígitos)
`tipo_cuenta`	`text`, `ahorro` o `corriente` (por defecto `ahorro`)
`moneda`	`text`, `PYG` o `USD` (por defecto `PYG`)
`saldo`	`numeric(14,2)`, obligatorio, por defecto 0; la base exige que sea mayor o igual a cero
`estado`	`text`, uno de `activa` / `bloqueada` / `cerrada`; por defecto `activa`
`activa`	`boolean`, obligatorio, por defecto `true`; `false` = dada de baja
`created_at`	`timestamptz`, por defecto `now()`

movimientos

Libro de la cuenta. Cada fila guarda el saldo que quedó después de aplicarla. Lo escriben este módulo (`manual`) y los de transferencias, ahorros y depósitos.

Columna	Tipo / restricción
`id`	`bigserial`, clave primaria
`cuenta_id`	`bigint`, obligatorio, referencia a `cuentas(id)`
`tipo`	`text`, `debito` o `credito`
`monto`	`numeric(14,2)`, obligatorio, la base exige que sea mayor que cero
`saldo_posterior`	`numeric(14,2)`, obligatorio: saldo de la cuenta inmediatamente después del movimiento
`referencia_tipo`	`text`, uno de `manual` / `transferencia` / `prestamo` / `ahorro` / `deposito` / `tarjeta`; por defecto `manual`
`referencia_id`	`bigint`, opcional: id de la operación que originó el movimiento
`descripcion`	`text`, opcional
`activo`	`boolean`, por defecto `true`; ningún endpoint lo cambia
`created_at`	`timestamptz`, por defecto `now()`

> El saldo solo cambia a través de un movimiento: no hay endpoint que lo edite directamente. El `saldo_posterior` del último movimiento de una cuenta debe coincidir con `cuentas.saldo`. En los datos sembrados esa igualdad se cumple, pero el extracto sembrado no incluye todas las transferencias sembradas: no es un libro contable completo.

4. Requerimientos funcionales

RF-G1-01 — Listar cuentas

`GET /api/v2/cuentas`

Devuelve las cuentas vigentes del sandbox, con la posibilidad de acotar por titular y por estado de negocio.

Entradas y validaciones

- `usuarioId` (opcional, por query): número entero positivo.
- `estado` (opcional, por query): `activa`, `bloqueada` o `cerrada`.

Reglas de negocio

- Devuelve únicamente cuentas no dadas de baja (`activa = true`), en cualquier estado de negocio: una cuenta `cerrada` sigue apareciendo.
- Los dos filtros se combinan: con ambos, devuelve las cuentas de ese titular que están en ese estado.
- Un titular sin cuentas, o un `usuarioId` inexistente, devuelve una lista vacía y no un error.
- Máximo 100 filas, ordenadas por identificador ascendente.

Respuesta esperada

- `200 OK` con `data` como arreglo de cuentas; cada elemento incluye `id`, `usuario_id`, `numero_cuenta`, `tipo_cuenta`, `moneda`, `saldo`, `estado`, `activa` y `created_at`.

Errores

Código	HTTP	Cuándo ocurre
`VALIDATION_ERROR`	400	`usuarioId` no es un entero positivo, o `estado` no es uno de los tres valores permitidos.

Fuente: `app/api/v2/cuentas/route.ts`

RF-G1-02 — Abrir una cuenta

`POST /api/v2/cuentas`

Abre una cuenta nueva para un cliente, con saldo cero y número de cuenta generado por el sistema.

Entradas y validaciones

- `usuarioId` (obligatorio): número entero positivo del titular.
- `tipoCuenta` (obligatorio): `ahorro` o `corriente`.
- `moneda` (obligatorio): `PYG` o `USD`.

Reglas de negocio

- El `numero_cuenta` no lo envía el cliente: lo genera el servidor al azar (10 dígitos) y debe ser único.
- La cuenta nace con `saldo = 0`, `estado = 'activa'` y `activa = true`. No se puede abrir con saldo: el primer saldo se carga con un depósito (RF-G1-09).
- El titular debe existir: un `usuarioId` inexistente es rechazado por la clave foránea.
- **No se valida que el titular esté activo ni que ya tenga otra cuenta igual**: un cliente dado de baja puede recibir una cuenta, y un mismo cliente puede tener varias cuentas del mismo tipo y moneda.

Respuesta esperada

- `201 Created` con `data` conteniendo la cuenta creada.

Errores

Código	HTTP	Cuándo ocurre
`VALIDATION_ERROR`	400	Falta un campo obligatorio, `tipoCuenta` o `moneda` tienen un valor fuera de la lista, o el `usuarioId` no existe.
`CONFLICT`	409	El número de cuenta generado al azar ya existía (caso extremadamente improbable; no hay reintento).

Fuente: `app/api/v2/cuentas/route.ts`

RF-G1-03 — Consultar una cuenta

`GET /api/v2/cuentas/{id}`

Devuelve el detalle completo de una cuenta puntual.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.

Reglas de negocio

- Devuelve la cuenta cuyo identificador coincide exactamente, en cualquier estado de negocio.
- Si no existe o está dada de baja, responde 404 con el mensaje "Cuenta no encontrada.".

Respuesta esperada

- `200 OK` con `data` conteniendo la cuenta completa.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe una cuenta vigente con ese identificador.
`VALIDATION_ERROR`	400	El identificador no es un entero positivo.

Fuente: `app/api/v2/cuentas/[id]/route.ts`

RF-G1-04 — Modificar una cuenta

`PUT /api/v2/cuentas/{id}`

Reemplaza el tipo y la moneda de una cuenta. El número, el saldo y el estado quedan fuera de este reemplazo.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.
- `tipoCuenta` (obligatorio): `ahorro` o `corriente`.
- `moneda` (obligatorio): `PYG` o `USD`.

Reglas de negocio

- Es un reemplazo: los dos campos son obligatorios aunque solo cambie uno.
- `numero_cuenta`, `saldo` y `estado` no forman parte del cuerpo; si se envían, se ignoran.
- **No se valida el estado ni el saldo**: se puede modificar una cuenta `bloqueada` o `cerrada`, y se puede cambiar la moneda de una cuenta con saldo y movimientos (el importe no se convierte, solo cambia la etiqueta).

Respuesta esperada

- `200 OK` con `data` conteniendo la cuenta ya actualizada.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe una cuenta vigente con ese identificador.
`VALIDATION_ERROR`	400	Falta un campo o tiene un valor fuera de la lista permitida.

Fuente: `app/api/v2/cuentas/[id]/route.ts`

RF-G1-05 — Dar de baja una cuenta

`DELETE /api/v2/cuentas/{id}`

Da de baja lógica una cuenta sin saldo, quitándola de los listados y consultas.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.

Reglas de negocio

- Una cuenta con saldo mayor que cero no puede darse de baja: responde 409 con el mensaje "No se puede eliminar una cuenta con saldo distinto de 0.". Primero hay que vaciarla con un retiro o una transferencia.
- Con saldo cero, marca `activa = false`; la fila y sus movimientos permanecen en la base.
- **No se valida el estado ni los productos asociados**: se puede dar de baja una cuenta `bloqueada`, y una cuenta con tarjetas, préstamos, planes de ahorro o depósitos vinculados.
- Repetir la baja sobre la misma cuenta responde 404: ya no es una cuenta vigente.

Respuesta esperada

- `204 No Content`, sin cuerpo.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe una cuenta vigente con ese identificador.
`CONFLICT`	409	La cuenta tiene saldo mayor que cero.
`VALIDATION_ERROR`	400	El identificador no es un entero positivo.

Fuente: `app/api/v2/cuentas/[id]/route.ts`

RF-G1-06 — Cambiar el estado de una cuenta

`PATCH /api/v2/cuentas/{id}/estado`

Mueve la cuenta entre sus estados de negocio: bloquearla, desbloquearla o cerrarla definitivamente.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.
- `estado` (obligatorio, en el cuerpo): `activa`, `bloqueada` o `cerrada`.

Reglas de negocio

- `cerrada` es un estado terminal: una cuenta cerrada no admite ningún cambio de estado y responde 409 con el mensaje "Una cuenta cerrada no puede cambiar de estado.".
- Para cerrar una cuenta su saldo debe ser cero; con saldo mayor que cero responde 409 con el mensaje "No se puede cerrar una cuenta con saldo distinto de 0.".
- Las demás transiciones son libres: `activa` → `bloqueada`, `bloqueada` → `activa`, y `activa` o `bloqueada` → `cerrada`.
- Pedir el estado en el que la cuenta ya está no es un error: responde 200 sin cambios (salvo en una cuenta cerrada, que responde 409).
- Una cuenta `bloqueada` o `cerrada` no admite depósitos, retiros ni transferencias, pero sigue siendo consultable.

Respuesta esperada

- `200 OK` con `data` conteniendo la cuenta con su nuevo `estado`.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe una cuenta vigente con ese identificador.
`CONFLICT`	409	La cuenta ya está cerrada, o se intenta cerrar una cuenta con saldo.
`VALIDATION_ERROR`	400	Falta `estado` o no es uno de los tres valores permitidos.

Fuente: `app/api/v2/cuentas/[id]/estado/route.ts`

RF-G1-07 — Consultar el saldo

`GET /api/v2/cuentas/{id}/saldo`

Devuelve una vista reducida de la cuenta, pensada para verificar el saldo sin leer la fila entera.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.

Reglas de negocio

- `ultimo_movimiento` es la fecha y hora del movimiento más reciente de la cuenta; es `null` si la cuenta nunca tuvo movimientos.
- El saldo devuelto es el mismo de RF-G1-03: no se recalcula a partir del extracto.

Respuesta esperada

- `200 OK` con `data` conteniendo `cuenta_id`, `numero_cuenta`, `moneda`, `saldo`, `estado` y `ultimo_movimiento`. El identificador viaja como `cuenta_id`, no como `id`.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe una cuenta vigente con ese identificador.
`VALIDATION_ERROR`	400	El identificador no es un entero positivo.

Fuente: `app/api/v2/cuentas/[id]/saldo/route.ts`

RF-G1-08 — Listar movimientos de una cuenta

`GET /api/v2/cuentas/{id}/movimientos`

Devuelve el extracto de la cuenta, con la posibilidad de ver solo débitos o solo créditos.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.
- `tipo` (opcional, por query): `debito` o `credito`.

Reglas de negocio

- Primero verifica que la cuenta exista y no esté dada de baja; si no, responde 404 en lugar de una lista vacía.
- Devuelve los movimientos de esa cuenta ordenados por identificador ascendente (el más antiguo primero), con un máximo de 100.
- Incluye los movimientos de cualquier origen: los manuales de este módulo y los generados por transferencias, aportes de ahorro y depósitos a plazo.
- No hay filtro por fecha ni por importe.

Respuesta esperada

- `200 OK` con `data` como arreglo de movimientos; cada elemento incluye `id`, `cuenta_id`, `tipo`, `monto`, `saldo_posterior`, `referencia_tipo`, `referencia_id`, `descripcion`, `activo` y `created_at`.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe una cuenta vigente con ese identificador.
`VALIDATION_ERROR`	400	El identificador no es un entero positivo, o `tipo` no es `debito` ni `credito`.

Fuente: `app/api/v2/cuentas/[id]/movimientos/route.ts`

RF-G1-09 — Registrar un depósito o un retiro

`POST /api/v2/cuentas/{id}/movimientos`

Registra un movimiento manual sobre la cuenta: un crédito (depósito) suma al saldo y un débito (retiro) resta. El saldo y el movimiento se actualizan juntos.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.
- `tipo` (obligatorio): `credito` (depósito) o `debito` (retiro).
- `monto` (obligatorio): número mayor que cero. Debe enviarse como número JSON, no como texto.
- `descripcion` (opcional): texto libre.

Reglas de negocio

- La cuenta debe estar en estado `activa`; si está `bloqueada` o `cerrada` responde 409 con el mensaje "La cuenta está bloqueada: no admite movimientos." (o `cerrada`).
- Un débito no puede superar el saldo: responde 409 con el mensaje "Saldo insuficiente para el débito solicitado.". Un débito por exactamente el saldo es válido y deja la cuenta en cero.
- Un crédito no tiene tope de importe.
- El movimiento queda con `referencia_tipo = 'manual'`, sin `referencia_id`, y con `saldo_posterior` igual al saldo resultante.
- La operación es atómica y bloquea la cuenta mientras dura: dos retiros simultáneos no pueden dejar el saldo en negativo.
- Ante cualquier rechazo, ni el saldo ni el extracto cambian.

Respuesta esperada

- `201 Created` con `data` conteniendo el movimiento creado y, anidada en `data.cuenta`, la cuenta con su saldo ya actualizado.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe una cuenta vigente con ese identificador.
`CONFLICT`	409	La cuenta no está activa, o el débito supera el saldo.
`VALIDATION_ERROR`	400	Falta `tipo` o `monto`, el `tipo` no es válido, o el `monto` es cero, negativo o no es un número JSON.

Fuente: `app/api/v2/cuentas/[id]/movimientos/route.ts`

RF-G1-TK — Obtener el token del grupo

`POST /api/v2/g1/auth/token`

Intercambia el usuario y la contraseña del grupo por un token JWT que puede usarse en lugar de la API key en cualquier ruta del curso. La ruta en sí también exige estar autenticado (API key del curso 2 o un token vigente).

Entradas y validaciones

- `username` (obligatorio): texto de 1 a 64 caracteres. El usuario del grupo es `c2_g01_cuentas`.
- `password` (obligatorio): texto de 1 a 200 caracteres. Se entrega por el canal del curso; no se publica en este documento.

Reglas de negocio

- La contraseña se compara en la base de datos contra un hash bcrypt; nunca se devuelve ni se registra en claro.
- Usuario inexistente, contraseña incorrecta y credencial inactiva devuelven el mismo `400 VALIDATION_ERROR` con el mensaje "Usuario o password inválidos.": la respuesta no revela cuál de los tres falló. Es `400` y no `401` a propósito, para no confundirlo con el fallo de la API key.
- Si las credenciales son válidas pero pertenecen a otro grupo, la respuesta es `403 FORBIDDEN` con el mensaje "Estas credenciales pertenecen al grupo 2 y esta ruta es del grupo 1.".
- El token es un JWT firmado con HS256, emitido por `aiquaa-sandbox`, con vigencia de **3600 segundos**. Sus claims (`sub`, `usuarioId`, `grupo`, `curso`) son legibles: el token está firmado, no cifrado.
- El token identifica al grupo pero no restringe por grupo: con él se puede operar sobre los recursos de cualquier módulo del curso 2.
- La emisión no deja rastro en tablas de dominio (el curso 2 no tiene tabla de sesiones); solo queda la fila de auditoría.

Respuesta esperada

- `200 OK` con `data` conteniendo `token`, `tokenType` (`Bearer`), `expiresIn` (`3600`), `grupo` (`1`), `grupoNombre`, `curso` (`2`) y `usuario` (`id`, `nombre`, `email`, `username`).

Errores

Código	HTTP	Cuándo ocurre
`VALIDATION_ERROR`	400	Falta `username` o `password`, están vacíos, o la combinación no es válida.
`FORBIDDEN`	403	Las credenciales son de otro grupo, o la API key usada para llamar es del curso 1.
`UNAUTHORIZED`	401	La request no trae API key ni token.

Fuente: `app/api/v2/g1/auth/token/route.ts`, `lib/group-token-route.ts` y `lib/jwt.ts`

5. Reglas transversales de la API

Todas las rutas REST comparten el mismo pipeline: autenticación → control de cohorte (curso) → control de caudal → validación de entrada → ejecución del SQL fijo → auditoría. Estas reglas aplican a cada requerimiento del documento y no se repiten en cada uno.

- **Envoltura de respuesta:** las respuestas exitosas devuelven `{ "data": ... }` y las fallidas `{ "error": { "code", "message", "details?" } }`.
- **Tipos en la respuesta:** los identificadores (`id`, `usuario_id`, `cuenta_id`, ...) viajan como **número**; los importes y tasas (`saldo`, `monto`, `limite_credito`, `tasa_anual`, ...) viajan como **texto con dos decimales** (por ejemplo `"5000000.00"`), por ser valores numéricos exactos de la base; las fechas sin hora viajan como `YYYY-MM-DD` y las marcas de tiempo (`created_at`, `fecha_pago`) en formato ISO 8601. Una aserción sobre un importe debe comparar contra texto o convertir a número antes.
- **Validación de entrada:** cada ruta define un esquema; los parámetros de query, el cuerpo JSON y los parámetros de ruta se combinan en un único objeto con precedencia query < body < path (un `{id}` de la URL siempre gana sobre un `id` del cuerpo). Los campos no declarados en el esquema se ignoran sin error.
- **Coerción de tipos:** los identificadores (`id`, `usuarioId`, `cuentaId`, ...) se convierten de texto a número automáticamente, vengan por query, por ruta o en el cuerpo. Los **importes, tasas y plazos del cuerpo JSON deben enviarse como número**: enviarlos como texto (`"monto": "1000"`) devuelve `400 VALIDATION_ERROR`.
- **Cuerpo JSON:** un cuerpo que no parsea como JSON devuelve `400 VALIDATION_ERROR` con el mensaje "Invalid JSON body."; un cuerpo vacío se interpreta como `{}`. Un fallo del esquema devuelve `400 VALIDATION_ERROR` con el mensaje "Invalid request." y el detalle del campo en `error.details`.
- **Reglas de negocio → 409:** cuando el cuerpo es válido pero el estado actual del recurso no permite la operación (saldo insuficiente, recurso en un estado que no admite la acción), la respuesta es `409 CONFLICT` con un mensaje en español que describe la regla. `400` queda para entradas inválidas.
- **Errores de base de datos, mapeados por código SQLSTATE:** una violación de unicidad (`23505`) se traduce a `409 CONFLICT`; una violación de clave foránea o de restricción `CHECK` (`23503` / `23514`, por ejemplo un `usuarioId` inexistente) se traduce a `400 VALIDATION_ERROR`; cualquier otro error de base de datos (una fecha imposible, un número fuera de rango) cae en `400 EXECUTION_ERROR`. En los tres casos el `message` es el texto crudo de Postgres.
- **Importes con más de dos decimales:** la API no valida la cantidad de decimales; la base redondea a dos al guardar.
- **Operaciones que mueven dinero:** se ejecutan en una transacción con bloqueo de las filas involucradas; si alguna validación falla, no queda ningún cambio parcial (ni saldo modificado ni movimiento registrado).
- **Listados:** devuelven como máximo 100 filas ordenadas por identificador ascendente, sin paginación ni indicación de que hubo recorte. Los filtros opcionales se combinan entre sí (`AND`).
- **Headers de caudal:** toda respuesta que pasó el control de caudal incluye `X-RateLimit-Limit`, `X-RateLimit-Remaining` y `X-RateLimit-Reset` (este último en milisegundos Unix).
- **Verbo no soportado:** una ruta que no define el método HTTP usado responde `405 Method Not Allowed`; lo genera el framework y su cuerpo no sigue la envoltura `{ error }` del resto de la API.
- **Auditoría:** las requests quedan registradas en `public.sql_audit_log` con la credencial usada, el método y la ruta, la entrada validada, el resultado y la IP de origen. Un fallo de auditoría nunca convierte una respuesta exitosa en error.
- **Sin control de titularidad:** la API no tiene roles ni noción de "mis" productos. Cualquier credencial válida del curso puede leer y modificar los recursos de cualquier cliente, y ningún endpoint verifica que la cuenta indicada pertenezca al `usuarioId` indicado.

Códigos de éxito

Status	Cuándo ocurre
`200 OK`	GET/PUT/PATCH exitoso, y las acciones que cambian un estado sin crear un registro (bloquear, activar, aprobar, pagar una cuota, anular, cancelar).
`201 Created`	POST que crea un recurso o registra un movimiento de dinero nuevo.
`204 No Content`	DELETE exitoso (baja lógica) — la respuesta no lleva cuerpo.

Códigos de error

Código	HTTP	Significado
`UNAUTHORIZED`	401	Faltan credenciales, la API key no existe o está inactiva, o el token es inválido o venció.
`FORBIDDEN`	403	La credencial es válida pero es de otro curso, o las credenciales de un grupo se usaron en la ruta de token de otro grupo.
`RATE_LIMITED`	429	Se superaron las 30 requests por minuto de esa credencial.
`VALIDATION_ERROR`	400	La entrada no cumple el esquema de la ruta, el JSON es inválido, o Postgres rechazó una clave foránea/CHECK.
`EXECUTION_ERROR`	400	La consulta falló en la base de datos por un motivo no cubierto por los códigos anteriores.
`NOT_FOUND`	404	El recurso indicado no existe o ya está dado de baja.
`CONFLICT`	409	Regla de negocio violada sobre un recurso que existe, o violación de una restricción `UNIQUE`.
`INTERNAL_ERROR`	500	Fallo inesperado del servidor (por ejemplo, no se pudo verificar la API key).

6. Flujo en la aplicación web

La aplicación web consume exactamente los mismos endpoints a través del proxy `/api/proxy/v2/{path}`, que agrega la API key del lado del servidor y reenvía los headers de límite de caudal. El menú muestra los módulos del grupo del alumno (más "Clientes", siempre visible) cuando su email figura en el roster del curso; si no figura, muestra todos. Cada listado arranca filtrado por el cliente con el que se inició sesión y permite cambiar a otro titular o a "todos"; la búsqueda, el orden y la paginación de los listados se resuelven en el navegador. **El front es deliberadamente más estricto que la API en varios puntos**: esas reglas están marcadas como "regla de UI" y solo se pueden probar por pantalla — la misma operación enviada directo a la API es aceptada. Los errores de la API se muestran en pantalla con el mensaje que devolvió el backend, y un `401` cierra la sesión local.

Pantalla	Qué permite hacer
`/v2/cuentas` — Listado de cuentas	Tabla con número, titular, tipo, moneda, saldo y estado; filtros por titular y por estado, búsqueda por número de cuenta y totales de saldo por moneda. Ejecuta RF-G1-01.
`/v2/cuentas/new` — Abrir cuenta	Formulario con titular, tipo de cuenta, moneda y aceptación del contrato. Ejecuta RF-G1-02 y navega al detalle de la cuenta creada.
`/v2/cuentas/{id}` — Detalle de cuenta	Muestra saldo, estado y titular; permite bloquear, desbloquear y cerrar (RF-G1-06), registrar un depósito o un retiro (RF-G1-09), editar tipo y moneda (RF-G1-04), ver el extracto con filtro por tipo (RF-G1-08) y eliminar la cuenta (RF-G1-05).

- **Regla de UI:** el formulario de apertura no deja abrir una segunda cuenta del mismo tipo y moneda para un cliente que ya tiene una no cerrada. La API sí lo permite.
- **Regla de UI:** la apertura exige marcar la aceptación del contrato; ese dato no se envía a la API.
- **Regla de UI:** la moneda solo se puede editar en una cuenta sin saldo ni movimientos, y una cuenta cerrada no se puede editar. La API acepta ambos cambios.
- **Regla de UI:** la descripción de un movimiento admite hasta 60 caracteres. La API no limita su longitud.
- El retiro por un importe mayor al saldo se rechaza en el navegador antes de llegar a la API, con el mensaje "Saldo insuficiente: el disponible es ...".
- Los botones "Cerrar cuenta" y "Eliminar cuenta" quedan deshabilitados mientras la cuenta tenga saldo; bloquear y cerrar piden confirmación.
- El extracto se muestra del más reciente al más antiguo, al revés del orden en que lo devuelve la API.

7. Criterios de aceptación

Criterios de aceptación redactados en prosa, uno o más por requerimiento, cubriendo el camino feliz y los caminos negativos. Son la base para derivar escenarios BDD.

RF-G1-01 — Listar cuentas

- Al listar sin filtros, la respuesta es 200 y `data` es un arreglo de como máximo 100 elementos ordenados por `id` ascendente.
- Al filtrar por un titular con cuentas, todos los elementos devueltos tienen ese `usuario_id`.
- Al filtrar por `estado=bloqueada`, todos los elementos devueltos tienen `estado` igual a `bloqueada`.
- Al combinar `usuarioId` y `estado`, cada elemento cumple las dos condiciones.
- Al enviar `estado=suspendida` o `usuarioId=abc`, la respuesta es 400 `VALIDATION_ERROR`.
- Una cuenta dada de baja con RF-G1-05 deja de aparecer en el listado.

RF-G1-02 — Abrir una cuenta

- Al abrir una cuenta para un titular existente, la respuesta es 201, `data.saldo` es `"0.00"`, `data.estado` es `activa` y `data.numero_cuenta` tiene 10 dígitos.
- La cuenta creada aparece de inmediato en RF-G1-01 filtrando por ese titular.
- Al abrir con `moneda=EUR` o `tipoCuenta=plazo`, la respuesta es 400 `VALIDATION_ERROR`.
- Al abrir para un `usuarioId` inexistente, la respuesta es 400 `VALIDATION_ERROR` y no se crea ninguna cuenta.

- Al abrir dos cuentas iguales (mismo titular, tipo y moneda), ambas respuestas son 201 con `numero_cuenta` distintos.

RF-G1-03 — Consultar una cuenta

- Dada una cuenta existente, al consultarla la respuesta es 200 y `data.id` coincide con el solicitado.
- Al consultar un identificador inexistente, la respuesta es 404 `NOT_FOUND` con el mensaje "Cuenta no encontrada.".
- Al consultar con `id=abc` o `id=0`, la respuesta es 400 `VALIDATION_ERROR`.
- Una cuenta en estado `cerrada` se puede consultar: la respuesta es 200.

RF-G1-04 — Modificar una cuenta

- Dada una cuenta `ahorro`, al enviar `tipoCuenta=corriente` con su misma moneda la respuesta es 200 y el cambio se refleja al volver a consultarla.
- El `numero_cuenta`, el `saldo` y el `estado` no cambian tras el reemplazo, aunque el cuerpo incluya esos campos.
- Al enviar solo `tipoCuenta` (sin `moneda`), la respuesta es 400 `VALIDATION_ERROR`.
- Al modificar una cuenta inexistente o dada de baja, la respuesta es 404 `NOT_FOUND`.
- Al cambiar la moneda de una cuenta con saldo, la respuesta es 200 y el `saldo` conserva el mismo valor numérico.

RF-G1-05 — Dar de baja una cuenta

- Dada una cuenta recién abierta (saldo 0), al darla de baja la respuesta es 204 sin cuerpo.
- Tras la baja, RF-G1-03, RF-G1-07 y RF-G1-08 sobre esa cuenta responden 404, y deja de aparecer en RF-G1-01.
- Dada una cuenta con saldo, al intentar darla de baja la respuesta es 409 `CONFLICT` y la cuenta sigue siendo consultable.
- Dar de baja dos veces la misma cuenta devuelve 204 la primera vez y 404 la segunda.

RF-G1-06 — Cambiar el estado de una cuenta

- Dada una cuenta `activa`, al enviar `estado=bloqueada` la respuesta es 200 y `data.estado` es `bloqueada`.
- Dada una cuenta `bloqueada`, al enviar `estado=activa` la respuesta es 200 y la cuenta vuelve a admitir movimientos.
- Dada una cuenta con saldo cero, al enviar `estado=cerrada` la respuesta es 200 y `data.estado` es `cerrada`.
- Dada una cuenta con saldo, al enviar `estado=cerrada` la respuesta es 409 `CONFLICT` y el estado no cambia.
- Dada una cuenta `cerrada`, cualquier cambio de estado (incluido `cerrada`) responde 409 `CONFLICT`.
- Dada una cuenta `activa`, al enviar `estado=activa` la respuesta es 200.
- Al enviar `estado=suspendida`, la respuesta es 400 `VALIDATION_ERROR`.

RF-G1-07 — Consultar el saldo

- Dada una cuenta recién abierta, la respuesta es 200, `data.saldo` es `"0.00"` y `data.ultimo_movimiento` es `null`.
- Después de un depósito, `data.saldo` refleja el importe depositado y `data.ultimo_movimiento` deja de ser `null`.
- El `saldo` de esta vista coincide con el `saldo` de RF-G1-03 y con el `saldo_posterior` del último movimiento de RF-G1-08.
- Al consultar el saldo de una cuenta inexistente, la respuesta es 404 `NOT_FOUND`.

RF-G1-08 — Listar movimientos de una cuenta

- Dada una cuenta recién abierta, la respuesta es 200 y `data` es un arreglo vacío.
- Después de un depósito y un retiro, `data` tiene dos elementos en el orden en que se registraron.
- Al filtrar por `tipo=credito`, todos los elementos devueltos tienen `tipo` igual a `credito`.
- Al enviar `tipo=ajuste`, la respuesta es 400 `VALIDATION_ERROR`.
- Al pedir los movimientos de una cuenta inexistente, la respuesta es 404 `NOT_FOUND` con el mensaje "Cuenta no encontrada.".

RF-G1-09 — Registrar un depósito o un retiro

- Dada una cuenta activa con saldo 0, al depositar 500000 la respuesta es 201, `data.tipo` es `credito`, `data.saldo_posterior` es `"500000.00"` y `data.cuenta.saldo` es `"500000.00"`.
- Dada una cuenta con saldo 500000, al retirar 200000 la respuesta es 201 y `data.saldo_posterior` es `"300000.00"`.
- Al retirar exactamente el saldo disponible, la respuesta es 201 y el saldo queda en `"0.00"`.
- Al retirar un importe mayor al saldo, la respuesta es 409 `CONFLICT` con el mensaje "Saldo insuficiente para el débito solicitado." y el saldo no cambia.
- Dada una cuenta `bloqueada`, tanto un depósito como un retiro responden 409 `CONFLICT`.
- Al enviar `monto` igual a 0, negativo o como texto (`"1000"`), la respuesta es 400 `VALIDATION_ERROR` y no se registra ningún movimiento.
- Dada una cuenta en dólares, al depositar 150.75 el saldo aumenta exactamente en 150.75, sin pérdida de centavos.
- El movimiento creado aparece como último elemento de RF-G1-08 con `referencia_tipo` igual a `manual`.

RF-G1-TK — Obtener el token del grupo

- Con el usuario y la contraseña del grupo 1, la respuesta es 200, `data.tokenType` es `Bearer`, `data.expiresIn` es 3600 y `data.grupo` es 1.
- Una request posterior a cualquier endpoint del módulo enviando solo `Authorization: Bearer <token>` (sin `x-api-key`) responde con éxito.
- Con una contraseña incorrecta, la respuesta es 400 `VALIDATION_ERROR` con el mensaje "Usuario o password inválidos."; con un usuario inexistente, la respuesta es idéntica.
- Con las credenciales válidas del grupo 2 sobre esta ruta, la respuesta es 403 `FORBIDDEN`.

- Con un token alterado o vencido en el header `Authorization`, cualquier endpoint responde 401 `UNAUTHORIZED`.

8. Anexo normativo (BCP / SEDECO) y brechas

> Este anexo es material de referencia orientativo para dar contexto de negocio a los escenarios de prueba. Las normas citadas del Banco Central del Paraguay (BCP) y de la Secretaría de Defensa del Consumidor y el Usuario (SEDECO) se mencionan de forma general y deben validarse con su texto vigente antes de usarse como criterio de cumplimiento. El sandbox es un entorno didáctico y no pretende cumplir ninguna de ellas.

Referencia normativa	Expectativa	Estado en el sandbox
Reglamentación del BCP sobre cuentas de depósito y cuentas básicas de ahorro	La apertura de una cuenta requiere identificar al titular y dejar constancia de las condiciones aceptadas.	Parcial: la cuenta queda asociada a un cliente identificado por documento, pero no se valida que esté activo y la aceptación del contrato existe solo en la pantalla web.
Ley 1015/97 y debida diligencia del cliente (SEPRELAD)	Conocer al cliente antes de operar y conservar el registro de sus operaciones para su monitoreo.	Parcial: cada movimiento queda registrado con importe, fecha y saldo resultante, pero no hay verificación de identidad, umbrales ni alertas por operaciones inusuales.
Marco del BCP sobre disponibilidad de fondos en cuentas de depósito	Un débito solo puede ejecutarse si la cuenta está operativa y tiene fondos suficientes.	Cubierto: un retiro exige cuenta `activa` y saldo suficiente, y la base impide el saldo negativo.
Ley 1334/98 de Defensa del Consumidor (SEDECO), deber de información	El usuario debe poder conocer su saldo y el detalle de los movimientos de su cuenta.	Cubierto: la API expone saldo y extracto, y la web los muestra en el detalle de la cuenta.
Normativa del BCP sobre cierre de cuentas y saldos remanentes	El cierre de una cuenta requiere liquidar su saldo y debe quedar registrado como definitivo.	Cubierto en la API (no se cierra con saldo y `cerrada` es terminal); los datos sembrados incluyen una cuenta cerrada con saldo que contradice la regla.

Brechas conocidas del sandbox

- La cuenta 10 de los datos sembrados está `cerrada` con saldo 45.000: un estado que `PATCH /estado` nunca permitiría alcanzar. Ese saldo queda inmovilizado, porque una cuenta cerrada no admite retiros.
- `PUT /cuentas/{id}` no mira el estado ni el saldo: permite modificar una cuenta cerrada y cambiar la moneda de una cuenta con saldo, sin convertir el importe.
- `DELETE /cuentas/{id}` no verifica productos asociados: una cuenta con tarjetas, préstamos, planes de ahorro o depósitos vinculados se puede dar de baja, y esos productos quedan apuntando a una cuenta que ya no se puede consultar.
- `POST /cuentas` no valida que el titular esté activo ni impide cuentas duplicadas del mismo tipo y moneda.
- No existe control de titularidad: cualquier credencial del curso puede depositar, retirar, bloquear o cerrar la cuenta de cualquier cliente.

- Los movimientos no se pueden corregir ni anular: no hay `PUT`, `DELETE` ni consulta por id de un movimiento individual.
- El extracto está limitado a los primeros 100 movimientos (los más antiguos), sin paginación ni filtro por fecha: en una cuenta con más de 100 movimientos, los más recientes no se pueden listar por la API.
- El extracto sembrado no es contablemente completo: hay transferencias sembradas en estado `completada` sin su movimiento correspondiente.
- No hay tope para un depósito, ni límites diarios de retiro, ni comisiones, ni intereses sobre el saldo.
- Pedir el estado en el que la cuenta ya está responde 200 en lugar de señalar que no hubo cambio.